

Competitive Programming Notebook

py.Sandu

Contents

1 DP	2	8.7 Bit2d	10
1.1 Coins Change	2		
1.2 Knapsack	2		
1.3 Longest Increasing Subsequence	2		
1.4 Kadane Algorithm	2		
2 Math	2		
2.1 Sieve Of Eratosthenes	2		
2.2 Factorial	3		
2.3 Modular Exponentiation	3		
2.4 Solve Linear System	3		
2.5 Matrix Fibonacci	3		
2.6 Extended Euclidean Algorithm	4		
2.7 Matrix Multiplication	4		
3 String	4		
3.1 Kmp	4		
4 Graph	4		
4.1 Floyd-warshall	4		
4.2 Dijkstra	5		
4.3 Mst Prim	5		
4.4 Eulerian Path	5		
4.5 Bfs	5		
4.6 Ford-fulkerson	6		
4.7 Mst Kruskal	6		
4.8 Dfs	6		
4.9 Bellman-ford	7		
5 Template	7		
5.1 Template	7		
6 Geometry	7		
6.1 Point	7		
6.2 Convex Hull	7		
6.3 Winding Number	7		
6.4 Cross Product	8		
6.5 Isleft	8		
7 General	8		
8 DS	8		
8.1 Trie	8		
8.2 Monotonic Stack	9		
8.3 Sparse table	9		
8.4 Bit	9		
8.5 Dsu	9		
8.6 Psum2d	10		

1 DP

1.1 Coins Change

```

1 // Find the minimum number of coins needed to make a
  given amount using a set of coin denominations
2 // Time complexity:  $O(n * m)$  where  $n$  is the amount
  and  $m$  is the number of coin denominations
3
4 int coinChange(const vector<int>& coins, int amount)
  {
5   vector<int> dp(amount + 1, INT_MAX);
6   dp[0] = 0;
7
8   for (int i = 1; i <= amount; i++) {
9     for (const int& coin : coins) {
10      if (coin <= i && dp[i - coin] != INT_MAX)
11      {
12        dp[i] = min(dp[i], dp[i - coin] + 1);
13      }
14    }
15
16    return dp[amount] == INT_MAX ? -1 : dp[amount];
17 }

```

1.2 Knapsack

```

1 // General take/not take dp problem
2 // Time Complexity: approx.  $O(n * m)$ 
3 // Space Complexity:  $O(n * m) / O(m)$ 
4 // Use cases: target sum(subset, partition), knapsack
5
6 // Top-down
7
8 const int MAXN = 1e3, MAXW = 1e3;
9
10 int knapsack_top_down(vector<int>& v, vector<int>& w,
  int x){
11   int n = (int)v.size();
12   vector<vector<int>>> dp(MAXN, vector<int>(MAXW,
  -1));
13   auto rec = [&](auto self, int i, int m) -> int {
14     if(i >= n || m == 0) return 0;
15     if(dp[i][m] != -1) return dp[i][m];
16     int not_take = self(self, i+1, m);
17     int take = 0;
18     if(m >= w[i]){
19       take = v[i] + self(self, i+1, m - w[i]);
20     }
21     return dp[i][m] = max(take, not_take);
22   };
23
24   return rec(rec, 0, x);
25 }
26
27 // Bottom-up
28
29 int knapsack_bottom_up(vector<int>& v, vector<int>& w,
  int x){
30   int n = (int)v.size();
31   vector<int> dp(x+1, 0);
32
33   for(int i=0; i < n; i++){
34     for(int m=x; m >= w[i]; m--){
35       dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
36     }
37   }
38
39   return dp[x];
40 }

```

1.3 Longest Increasing Subsequence

```

1 // Find the length of the longest increasing
  subsequence in a given array
2 // Time complexity:  $O(n \log n)$  where  $n$  is the length
  of the input array
3
4 int lis(vector<int>& arr) {
5   vector<int> dp;
6   for (int num : arr) {
7     auto it = lower_bound(dp.begin(), dp.end(),
  num);
8     if (it == dp.end()) {
9       dp.push_back(num);
10    } else {
11      *it = num;
12    }
13  }
14  return dp.size();
15 }

```

1.4 Kadane Algorithm

```

1 // Find maximum subarray sum using Kadane's algorithm
2 // Time complexity:  $O(n)$  where  $n$  is the length of the
  input array
3
4 int maxSubArraySum(const vector<int>& arr) {
5   int maxSum = arr[0];
6   int maxLocal = arr[0];
7
8   for (int i = 0; i < arr.size(); i++) {
9     maxLocal = max(arr[i], maxLocal + arr[i]);
10    maxSum = max(maxSum, maxLocal);
11  }
12
13  return maxSum;
14 }

```

2 Math

2.1 Sieve Of Eratosthenes

```

1 // Find all prime numbers up to a given limit using
  the Sieve of Eratosthenes algorithm
2 // Time complexity:  $O(n \log \log n)$  where  $n$  is the
  limit
3 // This idea can also be used to construct a table
  for the smallest prime factors,  $O(\log N)$  per
  query
4
5 vector<int> findPrimes(int limit) {
6   vector<bool> isPrime(limit + 1, true);
7   isPrime[0] = isPrime[1] = false;
8
9   for (int i = 2; i * i <= limit; i++) {
10    if (isPrime[i]) {
11      for (int j = i * i; j <= limit; j += i) {
12        isPrime[j] = false;
13      }
14    }
15  }
16
17  vector<int> primes;
18  for (int i = 2; i <= limit; i++) {
19    if (isPrime[i]) {
20      primes.push_back(i);
21    }
22  }
23  return primes;
24 }

```

```

25
26 vector<int> buildSPF(int limit){
27     vector<int> spf(limit+1);
28     iota(spf.begin(), spf.end(), 0);
29     for(int i = 2; i * i <= limit; i++) {
30         if(spf[i] == i){
31             for(int j = i * i; j <= limit; j += i) {
32                 if(spf[j] == j) spf[j] = i;
33             }
34         }
35     }
36
37     return spf;
38 }
39
40 vector<int> getFactors(vector<int>& spf, int x){
41     vector<int> factors;
42     while(x > 1){
43         factors.push_back(spf[x]);
44         x /= spf[x];
45     }
46
47     return factors;
48 }

```

2.2 Factorial

```

1 // Time Complexity: O(N) for precompute, O(1) for nCr
2 // Space Complexity: O(N)
3 // Use cases: calculating combinations, permutations,
4 // probabilities
5
6 const int MAXN, MOD;
7 vector<int> fact(MAXN), ifact(MAXN);
8
9 void precompute(){
10     fact[0] = 1; ifact[0] = 1;
11     for(int i=1; i<MAXN; i++){
12         fact[i] = (i * fact[i-1]) % MOD;
13     }
14
15     ifact[MAXN-1] = fexp(fact[MAXN-1], MOD-2, MOD);
16     for(int i=MAXN-2; i >= 1; i--){
17         ifact[i] = (ifact[i+1]*(i+1)) % MOD;
18     }
19 }
20
21 int nCr(int n, int r){
22     if(r < 0 or r > n) return 0;
23     int num = fact[n];
24     int den = (ifact[r] * ifact[n - r]) % MOD;
25     return (num * den) % MOD;
26 }

```

2.3 Modular Exponentiation

```

1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int fexp(int x, int b, int MOD){
6     int res = 1;
7
8     while(b > 0){
9         if(b & 1) res = (res * x) % MOD;
10        x = (x * x) % MOD;
11        b >>= 1;
12    }
13    return res;
14 }

```

2.4 Solve Linear System

```

1 // Solve a system of linear equations using Cramer's
2 // rule
3 // Time complexity: O(n^4) in the worst case
4 // Function to calculate the determinant of a matrix
5 double determinant(const vector<vector<double>>&
6     matrix) {
7     int n = matrix.size();
8     if (n == 1) {
9         return matrix[0][0];
10    }
11    double det = 0.0;
12    for (int col = 0; col < n; col++) {
13        vector<vector<double>> submatrix(n - 1,
14            vector<double>(n - 1));
15        for (int i = 1; i < n; ++i) {
16            for (int j = 0; j < n; ++j) {
17                if (j < col) {
18                    submatrix[i - 1][j] = matrix[i][j];
19                } else if (j > col) {
20                    submatrix[i - 1][j - 1] = matrix[
21                        i][j];
22                }
23            }
24            det += (col % 2 == 0 ? 1 : -1) * matrix[0][
25                col] * determinant(submatrix);
26        }
27    }
28    return det;
29 }
30
31 // Function to solve a system of linear equations
32 // using Cramer's rule
33 vector<double> solveLinearSystem(const vector<vector<
34     double>>& A, const vector<double>& b) // A is the
35     coefficient matrix and b is the constant vector
36 {
37     int n = A.size();
38
39     double detA = determinant(A);
40
41     vector<double> solution(n);
42     for (int i = 0; i < n; i++) {
43         vector<vector<double>> Ai = A;
44         for (int j = 0; j < n; ++j) {
45             Ai[j][i] = b[j];
46         }
47         solution[i] = determinant(Ai) / detA; //
48         Cramer's rule
49     }
50     return solution;
51 }

```

2.5 Matrix Fibonacci

```

1 // Calculate the Nth Fibonacci number using matrix
2 // exponentiation.
3 // Time Complexity: O(log N)
4 // Space Complexity: O(1)
5
6 void matrixExponentiation(vector<vector<long long>>&
7     matrix, int power) {
8     int n = matrix.size();
9     vector<vector<long long>> result(n, vector<long
10         long>(n, 0));
11
12     // Initialize result as the identity matrix
13     for (int i = 0; i < n; i++) {
14         result[i][i] = 1;
15     }
16 }

```

```

12     }
13
14     while (power) {
15         if (power % 2 == 1) {
16             matrixMultiplication(result, matrix,
17 result);
18         }
19         matrixMultiplication(matrix, matrix, matrix);
20         power /= 2;
21     }
22     matrix = result;
23 }

```

2.6 Extended Euclidean Algorithm

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that a*x + b*y =
   gcd(a, b)
3
4 int extendedEuclidean(int a, int b, int &x, int &y){
5     if(a == 0){
6         x = 0;
7         y = 1;
8         return b;
9     }
10
11     int x1, y1;
12     int gcd = extendedEuclidean(b % a, a, x1, y1);
13     x = x1;
14     y = x1 - (b / a) * y1;
15     return gcd;
16 }

```

2.7 Matrix Multiplication

```

1 // Multiply two matrices A and B, and return the
   result in matrix C.
2 // Time Complexity: O(n^3)
3 // Space Complexity: O(n^2)
4
5 vector<vector<int>>> matrixMultiplication(const vector
   <vector<int>>>& A, const vector<vector<int>>>& B) {
6     int n = A.size();
7     vector<vector<int>>> C(n, vector<int>(n, 0)); //
   Initialize result matrix with zeros
8
9     for (int i = 0; i < n; i++) {
10         for (int j = 0; j < n; j++) {
11             for (int k = 0; k < n; k++) {
12                 C[i][j] += A[i][k] * B[k][j];
13             }
14         }
15     }
16     return C;
17 }

```

3 String

3.1 Kmp

```

1 // Find all occurrences of a pattern in a given text
   using the Knuth-Morris-Pratt (KMP) algorithm
2 // Time complexity: O(n + m) where n is the length of
   the text and m is the length of the pattern
3
4 vector<int> LPS(const string& pattern) {
5     int m = pattern.size();
6     vector<int> lps(m, 0);
7     int len = 0; // length of the previous longest
   prefix suffix

```

```

8     int i = 1;
9
10    while (i < m) {
11        if (pattern[i] == pattern[len]) {
12            len++;
13            lps[i] = len;
14            i++;
15        } else {
16            if (len != 0) {
17                len = lps[len - 1];
18            } else {
19                lps[i] = 0;
20                i++;
21            }
22        }
23    }
24    return lps;
25 }
26
27 vector<int> KMP(const string& text, const string&
   pattern) {
28     vector<int> lps = LPS(pattern);
29     vector<int> occurrences;
30     int n = text.size();
31     int m = pattern.size();
32     int i = 0; // index for text
33     int j = 0; // index for pattern
34
35     while (i < n) {
36         if (text[i] == pattern[j]) {
37             i++;
38             j++;
39         }
40         if (j == m) {
41             occurrences.push_back(i - j); // Pattern
   found at index (i - j)
42             j = lps[j - 1]; // Continue to search for
   the next match
43         } else if (i < n && text[i] != pattern[j]) {
44             if (j != 0) {
45                 j = lps[j - 1];
46             } else {
47                 i++;
48             }
49         }
50     }
51     return occurrences;
52 }

```

4 Graph

4.1 Floyd-warshall

```

1 // Find the shortest paths in a weighted graph with
   positive or negative edge weights (but with no
   negative cycles).
2 // Time Complexity: O(V^3)
3 // Space Complexity: O(V^2)
4 // Use cases: reachability, minimax distance, global
   negative cycle detection(dist[i][i] < 0)
5
6 void floydWarshall(vector<vector<int>>& dist) { //
   dist(INF)
7
8     int n = dist.size() - 1;
9     for(int i = 1; i <= n; i++) dist[i][i] = 0;
10
11     for (int k = 1; k <= n; k++) {
12         for (int i = 1; i <= n; i++) {
13             for (int j = 1; j <= n; j++) {
14                 if (dist[i][k] != INF and dist[k][j]
   != INF) {

```

```

15         dist[i][j] = min(dist[i][j], dist
16         [i][k] + dist[k][j]);
17     }
18 }
19 }
20 }

```

4.2 Dijkstra

```

1 // Find shortest path from a source vertex to all
  // other vertices in a graph with non-negative edge
  // weights
2 // Time Complexity:  $O((V + E) \log V)$ 
3 // Space Complexity:  $O(V)$ 
4 // Use cases: shortest path in non-negative weighted
  // graphs
5
6 void dijkstra(vector<vector<pair<int, int>>>& adj,
  int src) {
7
8     int n = adj.size() - 1;
9     vector<int> dist(n+1, INT_MAX);
10    dist[src] = 0;
11
12    priority_queue<pair<int, int>, vector<pair<int,
  int>>, greater<pair<int, int>>> pq; // Min-heap
  priority queue
13    pq.push({0, src});
14
15    while (!pq.empty()) {
16
17        auto [d, u] = pq.top();
18        pq.pop();
19
20        if(d > dist[u]) continue;
21
22        for (auto p : adj[u]) {
23
24            int v = p.first;
25            int weight = p.second;
26
27            if (dist[u] != INT_MAX && dist[u] +
  weight < dist[v]) {
28                dist[v] = dist[u] + weight;
29                pq.push({dist[v], v});
30            }
31        }
32    }
33 }

```

4.3 Mst Prim

```

1 // Prim's algorithm for finding the Minimum Spanning
  // Tree
2 // Time complexity:  $O(E \log V)$  where E is the number
  // of edges and V is the number of vertices
3
4 struct Edge {
5     int u, v, weight;
6     bool operator<(const Edge& other) const {
7         return weight > other.weight; // Min-heap
8     }
9 };
10
11 vector<Edge> primMST(const vector<vector<int>>>& graph
  ) {
12     int n = graph.size();
13     vector<bool> visited(n, false);
14     vector<Edge> mst;
15     priority_queue<Edge> pq;
16

```

```

// Start with the first vertex
visited[0] = true;
for (int i = 0; i < n; ++i) {
    if (graph[0][i] != 0) {
        pq.push({0, i, graph[0][i]});
    }
}

while (!pq.empty()) {
    Edge current = pq.top();
    pq.pop();

    if (visited[current.v]) continue;

    visited[current.v] = true;
    mst.push_back(current);

    for (int i = 0; i < n; ++i) {
        if (!visited[i] && graph[current.v][i] !=
  0) {
            pq.push({current.v, i, graph[current.
  v][i]});
        }
    }
}

return mst;
}

```

4.4 Eulerean Path

```

1 // Find the Eulerian path in a graph using Hierholzer
  // 's algorithm
2 // Time complexity:  $O(E)$  where E is the number of
  // edges in the graph
3 // Use cases: visit every edge once, DNA sequencing,
  // puzzles, drawing figures
4
5 vector<int> findEulerianPath(vector<vector<int>>& adj
  , int start) {
6
7     vector<int> path;
8     stack<int> s;
9     s.push(start);
10
11     while (!s.empty()) {
12         int v = s.top();
13         if (adj[v].empty()) {
14             path.push_back(v);
15             s.pop();
16         } else {
17             int u = adj[v].back();
18             adj[v].pop_back(); // Remove the edge
  from v to u
19             s.push(u);
20         }
21     }
22
23     reverse(path.begin(), path.end());
24     return path;
25 }

```

4.5 Bfs

```

1 // Breadth First Search (BFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 void bfs(const Graph& graph, int start) {
6
7     int V = graph.size();
8     vector<bool> visited(V, false);

```

```

9     queue<int> q;
10
11     visited[start] = true;
12     q.push(start);
13
14     while (!q.empty()) {
15         int vertex = q.front();
16         q.pop();
17
18         for (int neighbor : graph[vertex]) {
19             if (!visited[neighbor]) {
20                 visited[neighbor] = true;
21                 q.push(neighbor);
22             }
23         }
24     }
25 }

```

4.6 Ford-fulkerson

```

1 // Find maximum flow in graph
2 // Time complexity:  $O(|V| * E^2)$ 
3 // Space complexity:  $O(V)$ 
4
5 bool bfs(int residual[V][V], int start, int target,
6          vi parent){
7     vector<bool> visited(V, false);
8
9     queue<int> q;
10    q.push(start);
11    visited[start] = true;
12    parent[start] = -1;
13
14    while (!q.empty()){
15        int u = q.front();
16        q.pop();
17
18        for(int v = 0 ; v <= V ; v++){
19            if(!visited[v] and residual[u][v] > 0){
20                if(v == target){
21                    parent[v] = u;
22                    return true;
23                }
24                q.push(v);
25                parent[v] = u;
26                visited[v] = true;
27            }
28        }
29    }
30    return false;
31 }
32
33 int fordFulkerson(int g[V][V], int start, int target) {
34     {
35         int u, v;
36         int residual[V][V];
37
38         for(u = 0 ; u < V ; u++)
39             for(v = 0 ; v < V ; v++)
40                 residual[u][v] = g[u][v];
41
42         vi parent(V);
43         int maxFlow = 0;
44
45         while(bfs(residual, start, target, parent)){
46             int pathFlow = INT_MAX;
47             for(v = target ; v != start ; v = parent[v]){
48                 u = parent[v];
49                 pathFlow = min(pathFlow, residual[u][v]);
50             }
51             for(v = target ; v != start ; v = parent[v]){

```

```

52                 residual[u][v] -= pathFlow;
53                 residual[v][u] += pathFlow;
54             }
55             maxFlow += pathFlow;
56         }
57         return maxFlow;
58     }

```

4.7 Mst Kruskal

```

1 // Kruskal algorithm for finding the Minimum Spanning
  // Tree
2 // Time Complexity:  $O(E \log E)$ 
3 // Space complexity:  $O(V + E)$ 
4 // Use cases: connect nodes with min/max cost,
  // clustering, minimax path
5
6 // edges: {weight, u, v}
7 pair<int, vector<vector<int>>> kruskalMST(vector<
  array<int, 3>>& edges, int n){
8     int m = (int)edges.size();
9     sort(edges.begin(), edges.end());
10    vector<vector<int>> mst(n+1);
11    DSU dsu(n+1); int cost = 0;
12    for(int i=0; i<m; i++){
13        int a = edges[i][1], b = edges[i][2];
14        if(dsu.merge(a, b)){
15            mst[a].push_back(b);
16            mst[b].push_back(a);
17            cost += edges[i][0];
18        }
19    }
20    return {cost, mst};
21 }

```

4.8 Dfs

```

1 // Depth First Search (DFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 // Recursive DFS
6 void dfs(const Graph& graph, int vertex, vector<bool>
  & visited) {
7     visited[vertex] = true;
8
9     for (int neighbor : graph[vertex]) {
10         if (!visited[neighbor]) {
11             dfs(graph, neighbor, visited);
12         }
13     }
14 }
15
16 // Iterative DFS
17 void dfsIterative(const Graph& graph, int start) {
18     int V = graph.size();
19     vector<bool> visited(V, false);
20     stack<int> s;
21     s.push(start);
22
23     while (!s.empty()) {
24         int vertex = s.top();
25         s.pop();
26
27         if (!visited[vertex]) {
28             visited[vertex] = true;
29
30             for (int neighbor : graph[vertex]) {
31                 if (!visited[neighbor]) {
32                     s.push(neighbor);
33                 }
34             }

```

```

35     }
36 }

```

4.9 Bellman-ford

```

1 // Find the shortest path from a source vertex to all
  // other vertices in a graph with negative weight
  // edges and detect negative weight cycles.
2 // Time Complexity: O(V * E)
3 // Space Complexity: O(V)
4 // Use cases: SSSP with negative edges, negative
  // cycle detection, system of difference constraints
  // , shortest path with at most K edges
5
6 void bellmanFord(vector<vector<pair<int, int>>>& adj,
  int src) {
7
8     int n = adj.size() - 1;
9     vector<int> dist(n+1, INF);
10    dist[src] = 0;
11
12    // Relax all edges n-1 times
13    for (int i = 1; i < n; i++) {
14        for (int u = 1; u <= n; u++) {
15            for (auto& [v, w] : adj[u]) {
16                if (dist[u] != INF and dist[u] + w <
17                    dist[v]) {
18                    dist[v] = dist[u] + w;
19                }
20            }
21        }
22
23        // Check for negative weight cycles
24        for (int u = 1; u <= n; u++) {
25            for (auto& [v, w] : adj[u]) {
26                if (dist[u] != INF and dist[u] + w < dist
27                    [v]) {
28                    cout << "Graph contains negative
29                    weight cycle\n";
30                    return;
31                }
32            }
33        }
34    }
35 }

```

5 Template

5.1 Template

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 // #define int long long
6 #define vi vector<int>
7 #define pii pair<int, int>
8 #define debug(x...) cout<<"["#x"]: ",[](auto...$){((
  cout<<$<<" "; ...);}(x),cout<<endl
9
10 void solve(){
11
12 }
13
14 signed main(){
15     ios::sync_with_stdio(false);
16     cin.tie(NULL);
17     int t = 1;
18     //cin >> t;
19     while(t-->0) solve();
20     return 0;
21 }

```

6 Geometry

6.1 Point

```

1 struct Point {
2     double x, y;
3 };

```

6.2 Convex Hull

```

1 // Returns a list of points on the convex hull
2 // in counter-clockwise order
3 vector<Point> convex_hull(vector<Point> A)
4 {
5     int n = A.size(), k = 0;
6
7     if (n <= 3)
8         return A;
9
10    vector<Point> ans(2 * n);
11
12    // Sort points lexicographically
13    sort(A.begin(), A.end());
14
15    // Build lower hull
16    for (int i = 0; i < n; ++i) {
17
18        // If the point at K-1 position is not a part
19        // of hull as vector from ans[k-2] to ans[k
20        -1]
21        // and ans[k-2] to A[i] has a clockwise turn
22        while (
23            k >= 2
24            && cross_product(ans[k - 2], ans[k - 1],
25                            A[i])
26                <= 0)
27            k--;
28            ans[k++] = A[i];
29        }
30
31    // Build upper hull
32    for (size_t i = n - 1, t = k + 1; i > 0; --i) {
33
34        // If the point at K-1 position is not a part
35        // of hull as vector from ans[k-2] to ans[k
36        -1]
37        // and ans[k-2] to A[i] has a clockwise turn
38        while (k >= t
39            && cross_product(ans[k - 2], ans[k -
40                            1],
41                            A[i - 1])
42                <= 0)
43            k--;
44            ans[k++] = A[i - 1];
45        }
46
47    // Resize the array to desired size
48    ans.resize(k - 1);
49
50    return ans;
51 }

```

6.3 Winding Number

```

1 // Check if a point is inside a polygon using the
  // winding number algorithm
2
3 bool isInsidePolygon(const Point& p, const vector<
  Point>& polygon) {
4     int windingNumber = 0;
5     int n = polygon.size();
6

```

```

7   for (int i = 0; i < n; i++) {
8       const Point& v1 = polygon[i];
9       const Point& v2 = polygon[(i + 1) % n];
10
11      if (v1.y <= p.y) {
12          if (v2.y > p.y && isLeft(p, v1, v2)) {
13              ++windingNumber;
14          }
15      } else {
16          if (v2.y <= p.y && !isLeft(p, v1, v2)) {
17              --windingNumber;
18          }
19      }
20  }
21
22  return windingNumber != 0;
23 }

```

6.4 Cross Product

```

1 double cross_product(Point O, Point A, Point B)
2 {
3     return (A.x - O.x) * (B.y - O.y)
4           - (A.y - O.y) * (B.x - O.x);
5 }

```

6.5 Isleft

```

1 // Check if a point is to the left of a segment
   defined by two points
2 bool isLeft(const Point& p, const Point& v1, const
   Point& v2) {
3     return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
   y) * (p.x - v1.x) > 0;
4 }

```

7 General

8 DS

8.1 Trie

```

1 // class for a node of the trie
2 class TrieNode {
3 public:
4     bool endofWord;
5     TrieNode* children[26];
6
7     // Constructore to initialize a trie node
8     TrieNode()
9     {
10         endofWord = false;
11         for (int i = 0; i < 26; i++) {
12             children[i] = nullptr;
13         }
14     }
15 };
16
17 // class for the Trie implementation
18 class Trie {
19 private:
20     TrieNode* root;
21
22 public:
23     Trie() { root = new TrieNode(); }
24
25     // Function to insert a word into the trie
26     void insert(string word)
27     {
28         TrieNode* node = root;

```

```

29         for (char c : word) {
30             int index = c - 'a';
31             if (!node->children[index]) {
32                 node->children[index] = new TrieNode
33             };
34             node = node->children[index];
35         }
36         node->endofWord = true;
37     }
38
39     // Function to search a word in the trie
40     bool search(string word)
41     {
42         TrieNode* node = root;
43         for (char c : word) {
44             int index = c - 'a';
45             if (!node->children[index]) {
46                 return false;
47             }
48             node = node->children[index];
49         }
50         return node->endofWord;
51     }
52
53     // Function to check if there is any word in the
   trie
54     // that starts with the given prefix
55     bool startsWith(string prefix)
56     {
57         TrieNode* node = root;
58         for (char c : prefix) {
59             int index = c - 'a';
60             if (!node->children[index]) {
61                 return false;
62             }
63             node = node->children[index];
64         }
65         return true;
66     }
67
68     // Function to delete a word from the trie
69     void deleteWord(string word)
70     {
71         TrieNode* node = root;
72         for (char c : word) {
73             int index = c - 'a';
74             if (!node->children[index]) {
75                 return;
76             }
77             node = node->children[index];
78         }
79         if (node->endofWord == true) {
80             node->endofWord = false;
81         }
82     }
83
84     // Function to print the trie
85     void print(TrieNode* node, string prefix) const
86     {
87         if (node->endofWord) {
88             cout << prefix << endl;
89         }
90         for (int i = 0; i < 26; i++) {
91             if (node->children[i]) {
92                 print(node->children[i],
93                     prefix + char('a' + i));
94             }
95         }
96     }
97
98     // Function to start printing from the root
99     void print() const { print(root, ""); }

```

100 };

8.2 Monotonic Stack

```
1 // Monotonic Stack, next greater number in a circular
  array
2 // Time Complexity: O(N)
3 // Space Complexity: O(N)
4 // Use cases: ring problems, finding next larger
  obstacle in a loop
5
6 int n; vector<int> arr(n);
7 vector<int> res(n, -1);
8 stack<int> st;
9
10 for (int i = 0; i < 2 * n; i++) {
11     int num = arr[i % n];
12     while (!st.empty() and arr[st.top()] < num) {
13         // remove os menores
14         res[st.top()] = num;
15         st.pop();
16     }
17     if (i < n) {
18         st.push(i);
19     }
20 }
21 // Monotonic Stack, next smaller element in a linear
  array
22 // Use cases: largest rectangle in histogram,
  subarray contribution
23
24 for (int i = 0; i < n; i++) {
25     while (!st.empty() and arr[st.top()] > arr[i
26 ]) { // remove os maiores
27         res[st.top()] = arr[i];
28         st.pop();
29     }
30     st.push(i);
31 }
```

8.3 Sparsetable

```
1 // Sparse Table
2 // Time Complexity: O(n * log n) build, O(1) querie
3 // Space complexity: O(n * log n)
4 // Idempotent operation f(a, a) = a (min, max, gcd,
  and, or)
5 // Use cases: idempotent functions range queries
6 struct SparseTable{
7
8     int n, p;
9     vector<vector<int>> st;
10
11     SparseTable(vector<int> &arr){
12         n = (int) arr.size();
13         p = __lg(n);
14         st.assign(p+1, vector<int>(n, 0));
15
16         for(int j=0; j<n; j++) st[0][j] = arr[j];
17         for(int i = 1; i <= p; i++){
18             int k = (1ll << (i-1));
19             int lim = n - 2*k + 1;
20             for(int j=0; j<lim; j++){
21                 st[i][j] = op(st[i-1][j], st[i-1][j+k
22 ]);
23             }
24         }
25
26         int query(int l, int r){
27             int len = r - l + 1;
```

```
28         int p_local = __lg(len), k = (1ll << p_local)
29         ;
30         return op(st[p_local][l], st[p_local][r - k +
31 1]);
32     }
33 }
```

8.4 Bit

```
1 // Binary Indexed Tree(BIT)
2 // Time complexity: query and update log N
3 // Space complexity: O(N)
4 // Use cases: dynamic psum, inversion counting,
  dynamic multiset
5
6 struct BIT{
7
8     int n; vector<int> bit;
9     BIT(int n) : n(n){
10         bit.assign(n+1, 0);
11     }
12
13     void update(int i, int x){
14         while(i <= n){
15             bit[i] += x;
16             i += i & -i;
17         }
18     }
19
20     int query(int i){
21         int sum = 0;
22         while(i > 0){
23             sum += bit[i];
24             i -= i & -i;
25         }
26         return sum;
27     }
28
29     int queryRange(int l, int r){
30         return query(r) - query(l-1);
31     }
32
33     // retorna o Índice do k-ésimo elemento da bit
34     int find_kth(int k){
35
36         int idx = 0;
37         int max_bit = __lg(n);
38
39         for(int i = max_bit; i>= 0; i--){
40             int next_idx = idx + (1ll << i);
41             if(next_idx <= n and bit[next_idx] < k){
42                 idx = next_idx;
43                 k -= bit[next_idx];
44             }
45         }
46
47         return idx+1;
48     }
49 };
50 }
```

8.5 Dsu

```
1 // Disjoint Set Union(DSU)
2 // Time Complexity: aprox. O(1)
3 // Space Complexity: O(n)
4 // Use cases: connected components, cycle detection,
  offline edge deletion
5
6 struct DSU{
7
```

```

8     vector<int> parent, size;
9
10    DSU(int n){
11        parent.assign(n+1, 0);
12        size.assign(n+1, 1);
13        iota(parent.begin(), parent.end(), 0);
14    }
15
16    int find(int v){
17        if(v == parent[v]) return v;
18        return parent[v] = find(parent[v]);
19    }
20
21    bool same(int a, int b){
22        return find(a) == find(b);
23    }
24
25    bool merge(int a, int b){
26        a = find(a);
27        b = find(b);
28        if(a != b){
29            if(size[a] < size[b]) swap(a, b);
30            parent[b] = a;
31            size[a] += size[b];
32            return true;
33        }
34        return false;
35    }
36
37 };

```

8.6 Psum2d

```

1 // Psum2D
2 // Time Complexity: Query O(1)
3 // Space Complexity: O(N*M)
4 // Use cases: static 2d range sum, fixed-size window
5 // search, max sum submatrix
6
7 int n, m;
8 vector<vector<int>> arr(n+1, vector<int>(m+1, 0));
9 vector<vector<int>> psum(n+1, vector<int>(m+1, 0));
10
11 for (int i = 1; i <= n; i++) {
12     for (int j = 1; j <= m; j++) {
13         cin >> arr[i][j];
14         psum[i][j] = arr[i][j]
15             + psum[i][j-1]
16             + psum[i-1][j]
17             - psum[i-1][j-1];
18     }
19 }

```

```

19
20
21 // canto superior esquerdo (x1, y1) canto inferior
22 // direito (x2, y2)
23 int x1, y1, x2, y2;
24 int res = psum[x2][y2]
25     - psum[x1 - 1][y2]
26     - psum[x2][y1 - 1]
27     + psum[x1 - 1][y1 - 1];

```

8.7 Bit2d

```

1 // Binary Indexed Tree(BIT) 2D
2 // Time complexity: query and update log N * log M
3 // Space complexity: O(N*M)
4 // Use cases: dynamic 2D prefix sums, counting active
5 // points in a dynamic rectangle
6
7 struct BIT2D {
8     int n, m; vector<vector<int>> bit;
9     BIT2D(int n, int m) : n(n), m(m) {
10         bit.assign(n+1, vector<int>(m+1, 0));
11     }
12
13     void update(int x, int y, int val) {
14         for (int i = x; i <= n; i += i & -i) {
15             for (int j = y; j <= m; j += j & -j) {
16                 bit[i][j] += val;
17             }
18         }
19     }
20
21     int query(int x, int y) {
22         int sum = 0;
23         for (int i = x; i > 0; i -= i & -i) {
24             for (int j = y; j > 0; j -= j & -j) {
25                 sum += bit[i][j];
26             }
27         }
28         return sum;
29     }
30
31     int queryRange(int x1, int y1, int x2, int y2) {
32         return query(x2, y2)
33             - query(x1 - 1, y2)
34             - query(x2, y1 - 1)
35             + query(x1 - 1, y1 - 1);
36     }
37 };
38

```